AUTOMATED RAILWAY INSPECTION

Trackside Wagon Inspection

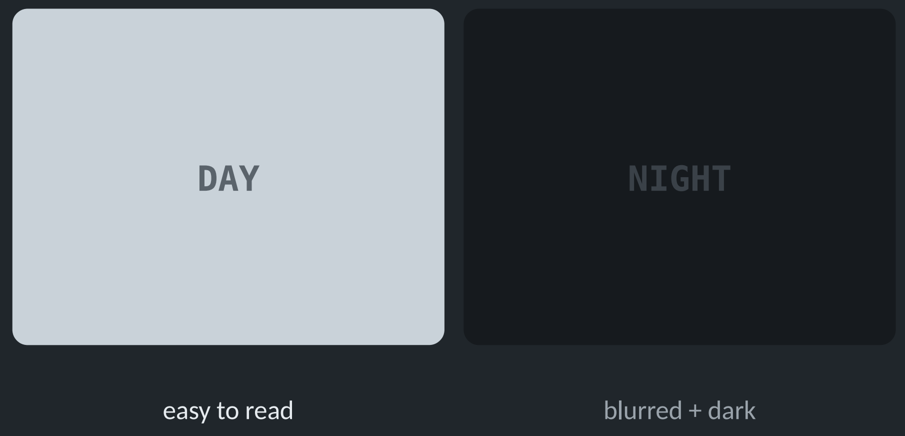
That Works When the Picture Is Bad

- Reads wagon numbers and finds defects on moving trains
- Runs at the trackside on a small edge device
- Sends text, not video — works on a 4G link
- Built for night, tunnels, rain, and camera shake
- Live dashboard for the control room

THE PROBLEM

Inspection Matters Most Exactly When the Image Is Worst

- Wagons must be checked as the train moves past
- People can’t read numbers on fast, dark wagons
- Night, tunnels, rain, and shake ruin the picture
- A missed crack or wrong number is a safety risk
- This happens at thousands of checkpoints, every day



This is the real operating condition.

WHY EXISTING APPROACHES FAIL

Most Systems Assume a Good Picture. Reality Doesn't Cooperate.

Time-based night mode

Wrong in a tunnel at noon, wrong in a floodlit yard at midnight.

Send video to a server

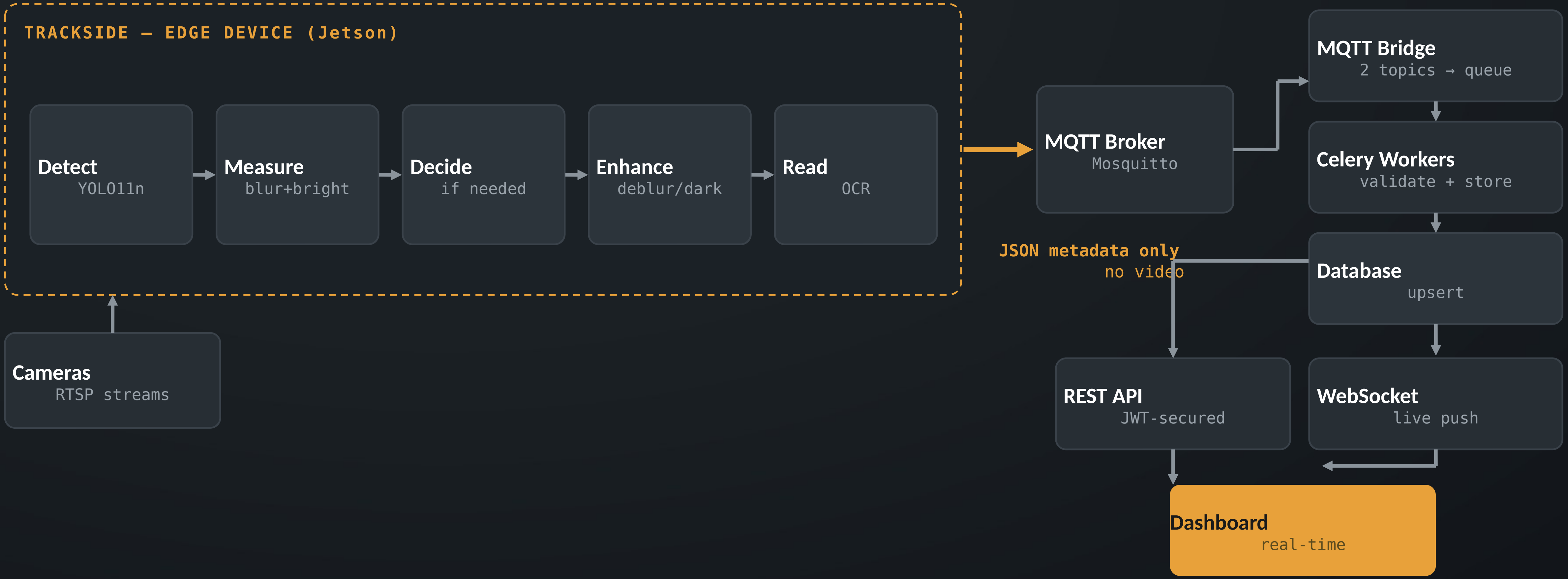
A trackside 4G link cannot carry a video stream.

Trust one sharp frame

OCR can be very confident and still read the wrong character.

We designed around all three.

Do the Hard Work at the Track. Send Only Text.



The boundary is the design. Heavy compute at the edge, a few hundred bytes over the network.

Five Steps, In Order, On One Small Device

Read live camera streams
Several cameras at once, over RTSP

Detect each wagon
Fast, lightweight model (YOLO11n)

Measure blur and brightness
Two simple numbers per frame

Read the wagon number
OCR on the detected region

Publish a tiny JSON message
Then move on to the next frame

THE QUALITY MEASUREMENT

This middle step is what makes the next decision possible.

Blur
`variance of the Laplacian — low = blurry`

Brightness
`mean grey level — low = dark`

React to the Actual Image, Not the Clock

Frame + OCR confidence
check the actual signal

Confident? → **Skip** save GPU + time

Blurry? → **Deblur** MIMO-UNet

Dark? → **Low-light** Retinex

Enhancement is expensive

So it only runs when the reading is actually poor. Because the trigger is measured quality, the same logic is correct in a bright tunnel exit and a dark midnight yard.

Trigger = measured quality, never the time of day.

We Show a Number, We Don't Make a Claim

RAW FRAME		ENHANCED FRAME	MEASURED GAIN
N4021	enhance →	N4021	+0.36
confidence 0.55		confidence 0.91	stored for every frame

- Run OCR twice — raw frame and enhanced frame
- Store both readings and both confidence scores
- Server computes the exact percentage gain
- If enhancement doesn't help, the number says so

Agreement Across Frames Beats One Confident Guess

CONSENSUS VOTING

frame 1	N4021
frame 2	N4021
frame 3	N4021
frame 4	N4021
frame 5	N4021

→ Consensus N4021

Kills the “99% sure but wrong” single-frame failure.

DEFECT DEDUPLICATION + HEALTH SCORE

One crack seen in 30 frames is charged once — deduplicated by component, defect type, and position zone.

Health starts at 100, subtracts per unique defect

−40 crack	−30 missing bolt
−15 bent	−10 foreign object
−5 rust	<85 inspect <60 critical

BACKEND ARCHITECTURE

From Tiny Messages to a Live Dashboard

MQTT Bridge

listens on 2 topics,
queues each message

→

Celery Workers

validate + store,
off the hot path

→

Redis + Celery

absorb bursts,
no dropped data

→

WebSocket

push results to
the dashboard live

→

REST API (JWT)

history + analytics
for later review

Why a queue in the middle? A spike in train traffic can't block ingestion — workers drain the queue at their own pace, so nothing is lost and the dashboard stays live.

The Track Is Messy. The System Assumes That.

Backpressure

If processing falls behind, drop the oldest frame, not the newest — live inspection needs "right now".

Idempotent writes

A replayed message updates its row instead of creating a twin. (camera, timestamp) is unique.

Capture ≠ receive time

Stored separately, so a 10-minute buffer during an outage never lies about when a frame was captured.

Graceful degradation

Missing model, OCR library, or broker → log it and fall back to classical processing, don't crash.

Fault tolerance

Auto-reconnect with backoff for cameras and broker; runs under systemd, restarts on failure.

RESULTS + CURRENT LIMITATIONS

What Works Today — and What Doesn't Yet

MEASURED IN OUR OWN TESTS

Wagon-number accuracy, raw → enhanced

[MEASURED: __%] → [MEASURED: __%]

Consensus reduced single-frame misreads by

[MEASURED: __%]

End-to-end latency, capture → dashboard

[MEASURED: __ ms]

Known limitations

- Defect model tested on a limited set of defect types.
- Not yet validated across seasons, weather, and multiple sites.

These are the first things on the roadmap.

Improvement from enhancement and consensus is measured, not assumed — ask me how.

More Cameras, More Sites, Same Small Messages

Edge site → text

Edge site → text

Edge site → text

Central backend
broker → workers →
DB → dashboard

Maintenance scheduling
inspection drives action, not just a dashboard

- Each edge device is independent — network cost per site stays tiny
- Broker and workers scale horizontally under load
- Expand defect types with more labelled field data
- Validate across seasons, weather, and multiple checkpoints